DEVELOPER REFERENCE

HYPERLIQUIDFLOW

Full technical specification: pipeline architecture, stream contracts, signal recipes, data model, scoring formulas, KV cache design, and infrastructure configuration.

VERSION	DATE	STACK	PLATFORM
1.0	April 2026	Next.js 15, Supabase, Vercel KV	Vercel (Hobby tier)

CONTENTS

Table of Contents

1 Architecture Overview 3
2 Data Model 4
3 Cohort Pipeline: Streams A–D 6
4 Signal Detection 8
5 Quality Assurance: Streams E–F 10
6 Infrastructure 11
7 API Routes 12
8 Known Limitations 13
9 Glossary 14

Document Scope

This document is the authoritative technical reference for the HyperliquidFLOW system. It covers every layer from raw API ingestion to client polling. Intended audience: engineers building on, extending, or operating the system.

The companion consumer whitepaper omits stream labels, recipe identifiers, database table names, formulas, and code. This document includes all of them.

Repository Layout

```
hyperliquidflow/
  app/               Next.js App Router pages and API routes
  components/        Shared React components (nav, cards, etc.)
  lib/               Core engines and utilities
  lib/__tests__/     Vitest unit tests
  scripts/           CLI scripts for daily scan and learning jobs
  supabase/          SQL migrations
  docs/              Whitepapers and sprint documentation
  .github/workflows/ GitHub Actions (daily scan, learning, freshness check)
```

SECTION 01

Architecture Overview

HyperliquidFLOW is a Next.js 15 App Router application. All heavy computation runs server-side or in background jobs. The browser receives pre-computed snapshots via a React Query polling loop.

Full Data Flow

```
Vercel Cron (00:00 UTC daily)      GitHub Actions (daily 00:00 UTC)
  /api/refresh-cohort                scripts/daily-wallet-scan.ts
  (seed + signal detection only)     +-- discover (leaderboard + fills)
  Hobby plan: must finish in 10s     +-- Stream A: activation gates (9 checks)
                                     +-- Stream C: wash / sybil detection
                                     +-- Stream D: behavior profiling
                                     +-- backtests + full scoring
                                     +-- writes Supabase + artifact

GitHub Actions ping (24/7)          GitHub Actions (01:00 UTC daily)
  keeps signal detection live         scripts/signal-learning.ts
  by calling /api/refresh-cohort      updates signal_outcomes stats

Browser (React)
  useQuery("/api/cohort-state") every 60s
    +-- read KV snapshot (fast path, ~5ms)
    +-- fires background refresh via after() if stale > 5 min
    +-- fallback to Supabase on KV miss
```

Pipeline Streams

STREAM	NAME	RUNTIME	PURPOSE
A	Activation Gates	daily-wallet-scan.ts	9-point quality gate for candidate wallets; writes wallets table
B	Cohort Hygiene	refresh-cohort API	Deactivates wallets that go quiet, blow up, or stop trading
C	Wash / Sybil	daily-wallet-scan.ts	Detects wash trading and Sybil address clusters
D	Behavior Profile	daily-wallet-scan.ts	Per-wallet style, conviction, and regime tendency
E	Signal Validation	refresh-cohort API	Filters emitted signals before they enter the KV snapshot
F	Signal Learning	signal-learning.ts	Updates outcome stats; drives recipe calibration

Core Engines

LIB/COHORT-ENGINE.TS

Four-factor wallet scoring: `sharpe_proxy`, `pnl_consistency`, `drawdown_score`, `regime_fit`. Exports `scoreWallet()`, `detectRegime()`, `pruneUnderperformers()`.

LIB/SIGNAL-LAB.TS

13 pluggable signal recipes. Each implements `(pair: SnapshotPair) ⇒ SignalEvent[]`. All registered in `runSignalLab()`.

LIB/RISK-ENGINE.TS

EV calculation, liquidation price, margin ratio, Hyperliquid fee schedule. Fee: 0.035% taker, 0.01% maker.

LIB/HYPERLIQUID-API-CLIENT.TS

Raw Hyperliquid API calls: clearinghouse states, market data, fill history. All calls POST to `https://api.hyperliquid.xyz/info`.

SECTION 02

Data Model

Supabase (PostgreSQL) stores all historical and scored data. Vercel KV caches hot snapshots for sub-10ms serving. Migrations live in `supabase/migrations/`.

Supabase Tables

wallets

One row per discovered address. Written by Stream A during the daily scan.

COLUMN	TYPE	NOTES
address	text PK	Hyperliquid address (0x...)
is_active	boolean	True if in current cohort
entity_type	text	human | bot | fund | unknown (migration 005)
score	numeric	Composite 0–100; updated daily
equity_tier	text	elite | major | large | mid | small | micro | dust
first_seen_at	timestamptz	Discovery timestamp
last_active_at	timestamptz	Last trade time from fills API
deactivation_reason	text	Null if active; Stream B exit code if deactivated

user_pnl_backtest

Daily realized PnL rows per wallet. Source of truth for scoring. Never conflate with unrealized positions in `cohort_snapshots`.

COLUMN	TYPE	NOTES
address	text FK	References wallets.address
date	date	UTC date of the PnL snapshot
daily_pnl	numeric	Realized PnL in USD for that day
cumulative_pnl	numeric	Running total from first observation

cohort_snapshots

Live positions per wallet captured each refresh cycle. Powers Stream E signal detection.

COLUMN	TYPE	NOTES
address	text FK	
snapshot_at	timestamptz	Time of capture
positions	jsonb	Array of open position objects from clearinghouse
account_value	numeric	Total margin account value at snapshot time

Signal Intelligence Tables (migrations 006-008)

`signals_history` stores every emitted `SignalEvent`. `signal_outcomes` drives the Stream F daily learning loop. `recipe_performance` holds rolling accuracy stats per recipe. `recipe_calibration` stores per-recipe threshold overrides tuned from outcome data.

Stream Output Tables

`wallet_hygiene` records Stream B deactivation events. `wash_sybil` stores cluster membership and wash-trade scores from Stream C. `wallet_profiles` stores the behavior style labels from Stream D via `lib/wallet-profile.ts`.

Vercel KV Cache Keys

KEY	TTL	CONTENT
cohort:active	120s	Main cohort snapshot: scored wallets + current signal events
cohort:active:fallback	24h	Stale backup written alongside primary; served on primary miss
cohort:cycle_offset	25h	Integer rotating window offset for partial cron refresh cycles
btc_mid:{HH}	25h	BTC mid price keyed by UTC hour; used for regime detection
market:prior_mids	10min	Map of coin to prior mid price; used by intraday signal recipes
market:funding_history:{coin}	25h	Recent funding rate array for a given coin
cohort:underwater_counts	25h	Per-coin count of wallets with losing open positions
recipe:intraday_perf	7h	Running recipe accuracy stats within the current trading day
market-ticker:v4	short	Live price and 24h change data for ticker strip
contrarian:latest	120s	Contrarian signal cache for /api/contrarian

Fallback Chain

```
// lib/server/kv-fetchers.ts - fetchCohortState()
const primary = await kv.get("cohort:active");
if (primary) return primary;

const fallback = await kv.get("cohort:active:fallback");
if (fallback) return fallback;

// Full Supabase read as last resort
return await supabase
  .from("cohort_snapshots")
  .select("*")
  .order("snapshot_at", { ascending: false })
  .limit(1);
```

TypeScript Interfaces

```
interface CohortSnapshot {
  wallets:         ScoredWallet[];
  signals:         SignalEvent[];
  updated_at:      number;         // Unix ms
  regime:          "BULL" | "BEAR" | "RANGING";
  btc_24h_pct:     number;
```

```
}

interface ScoredWallet {
  address:         string;
  score:           number;         // 0-100 composite
  sharpe_proxy:    number;
  pnl_consistency: number;
  drawdown_score:  number;
  regime_fit:      number;
  equity_tier:     string;
  is_active:       boolean;
}

interface SignalEvent {
  recipe_id:    string;   // e.g. "R5"
  wallet:       string;
  coin:         string;
  direction:    "LONG" | "SHORT" | "FLAT";
  signal_type:  "entry" | "exit" | "scale_in" | "scale_out" | "flip" | "alert";
  ev_score:     number;
  notional_usd: number;
  ts:           number;   // Unix ms
}

type SignalRecipe = (pair: SnapshotPair) => SignalEvent[];

interface SnapshotPair {
  prior:   CohortSnapshot;
  current: CohortSnapshot;
  regime:  "BULL" | "BEAR" | "RANGING";
}
```

SECTION 03

Cohort Pipeline: Streams A–D

The daily wallet scan runs at 00:00 UTC via GitHub Actions. It processes up to 5,000 candidate wallets and produces a scored cohort of approximately 500 active wallets. Runtime budget: 50 minutes.

A Activation Gates

Candidates must pass all 9 gates to enter the active cohort. Gate checks run in `scripts/daily-wallet-scan.ts` against 30 days of fill history and clearinghouse data.

GATE	FIELD / CHECK	THRESHOLD	NOTES
G1	win_rate	>= 0.52	Ratio of winning days to total trading days
G2	trade_count_30d	>= 10	Minimum 10 trades in 30-day window
G3	avg_notional	>= 500 USD	Filters dust-level wallets
G4	max_drawdown_30d	<= 0.40	Drawdown from equity peak; blocks blow-ups
G5	pnl_consistency	>= 0.3	Winning-day ratio over 30-day window
G6	active_days_30d	>= 5	Must have traded on at least 5 of last 30 days
G7	wash_score	<= 0.3	Stream C output; high score = likely wash trading
G8	sybil_cluster_size	<= 3	Stream C output; large clusters excluded
G9	entity_type	!= "bot"	Automated bots excluded from cohort

B Cohort Hygiene

Stream B runs inside `/api/refresh-cohort` on every cron cycle. It calls `pruneUnderperformers()` from `lib/cohort-hygiene.ts` and writes `wallet_hygiene` rows on deactivation.

Deactivation triggers: 5 consecutive quiet days with no trades, drawdown exceeding 50% from equity peak, or account value dropping below the dust threshold. The exit code is written to `wallets.deactivation_reason`.

Scoring Formula

```
// lib/cohort-engine.ts - scoreWallet()
overall_score =
  0.35 * sharpe_proxy      // risk-adjusted return
+ 0.25 * pnl_consistency   // rolling Sharpe stability
+ 0.25 * drawdown_score    // 1 - max_drawdown_30d
+ 0.15 * regime_fit        // accuracy in current regime
```

```
// Regime detection - detectRegime()
btc_24h_pct > +1.0  ⟹ "BULL"
btc_24h_pct < -1.0  ⟹ "BEAR"
otherwise           ⟹ "RANGING"

// sharpe_proxy
sharpe_proxy = mean(daily_pnl_30d) / stddev(daily_pnl_30d)

// pnl_consistency
pnl_consistency = winning_days / total_days  (30-day window)

// drawdown_score
drawdown_score = 1 - (peak_equity - current_equity) / peak_equity
```

C Wash Trading and Sybil Detection

Implemented in `lib/wash-sybil.ts`. Runs after discovery in `daily-wallet-scan.ts`. Output written to the `wash_sybil` Supabase table.

WASH TRADING DETECTION

Computes a `wash_score` per wallet from 0.0 to 1.0. High scores indicate self-trading patterns.

- Identifies opposing fills against the same counterparty within a short time window
- Flags fill sequences that produce no net position change
- Weights by recency; older patterns decay

SYBIL CLUSTER DETECTION

Groups wallets into clusters based on correlated fill timing and shared counterparty patterns.

- Cluster size written to `wash_sybil.cluster_size`
- Gate G8 blocks clusters of 4 or more
- Cluster membership stored in `wash_sybil.cluster_id`

D Behavior Profiling

Implemented in `lib/wallet-profile.ts`. Runs after scoring. Writes to `wallet_profiles`.

FIELD	TYPE	DESCRIPTION
`trading_style`	text	scalper | swing | position; derived from avg hold duration
`conviction_level`	numeric	0-1; ratio of large-notional trades vs total
`regime_tendency`	text	bull_hunter | bear_hunter | range_trader; win-rate per regime
`avg_hold_hours`	numeric	Average hours between entry fill and closing fill
`preferred_coins`	text[]	Top 3 coins by notional volume in 30-day window

Discovery Sources

Candidates are sourced from two paths each daily run:

- **Leaderboard:** top 2,000 addresses from the Hyperliquid PnL leaderboard API
- **Fill graph:** first-degree fill counterparties of existing cohort members (up to 3,000 additional candidates)

Deduplication runs before gate checking. Existing wallets in the `wallets` table are updated in place. Net new candidates that pass all gates are activated with `is_active = true`. At the end of each run, `scan-summary.json` is uploaded as a GitHub Actions artifact with 7-day retention.

SECTION 04

Signal Detection

Signal detection runs inside `/api/refresh-cohort` on every cron cycle. It calls `runSignalLab(pair)` with the prior and current cohort snapshot pair and collects all emitted `SignalEvent` objects. All 13 recipes run on every cycle.

Recipe Catalog

13 SIGNAL RECIPES - LIB/SIGNAL-LAB.TS

R1	**Liquidation Flush** Cohort wallet just got liquidated on a coin; contrarian entry signal for the opposite direction.	1-cycle
R2	**Hot Streak** Wallet has won 5+ consecutive days; flags current open positions as momentum candidates.	5d streak
R3	**Regime Shift** BTC crosses the BULL/BEAR threshold; repositioning signal for regime-sensitive wallets.	2-cycle
R4	**Size Up** High-conviction wallet meaningfully increases position notional in a single cycle.	1-cycle
R5	**Contrarian** Wallet opens a position that runs counter to the current majority direction in that coin.	1-cycle
R6	**Crowded Entry** 3+ top-quartile wallets enter the same coin in the same direction within one cycle.	1-cycle
R7	**Funding Flip** Funding rate sign changes for a coin where cohort wallets hold large open positions.	24h
R8	**Spread Capture** Wallet opens simultaneous long and short on correlated coins; statistical arb pattern.	1-cycle
R9	**Velocity Break** Notional velocity spikes 3x above the wallet's 7-day average within a single cycle.	7d baseline
R10	**Drawdown Recovery** Wallet was in drawdown > 20% last cycle; opens new positions this cycle.	2-cycle
R11	**Momentum Add** Wallet scales into an existing winning position after at least 2h of positive price movement.	2h window
R12	**Quiet Accumulation** Wallet builds position size across 3+ consecutive cycles with no public signal activity.	3-cycle

R13 **Regime Exit** 2-cycle

Top-quartile wallets close positions at the start of a regime change; precursor to reversal.

EV Scoring Formula

Every `SignalEvent` carries an `ev_score` computed by `lib/risk-engine.ts`. The score represents estimated expected value in USD after accounting for Hyperliquid fee costs.

```
// lib/risk-engine.ts - calculateEV()
trade_cost = (taker_fee_rate * ref_notional) + walkBook(depth_snapshot, ref_notional)

// Hyperliquid fee schedule:
//   taker_fee_rate = 0.00035  (0.035%)
//   maker_fee_rate = 0.0001   (0.010%)

ev = (win_prob * avg_win_usd) - ((1 - win_prob) * avg_loss_usd) - trade_cost

// win_prob        - from recipe_calibration table; updated daily by Stream F
// avg_win_usd     - rolling 30-day average per recipe
// avg_loss_usd    - rolling 30-day average per recipe
// walkBook()      - slippage estimate from order book depth snapshot
// ref_notional    - position notional in USD
```

Recipe Configuration

Per-recipe thresholds are stored in `lib/recipe-config.ts`. Stream F writes calibration updates to `recipe_calibration`; the config module merges these at runtime.

```
// lib/recipe-config.ts (example entries)
export const recipeConfig: RecipeConfigMap = {
  R1: {
    min_liquidation_usd:  10_000,
    lookback_cycles:      1,
    min_wallet_score:     60,
  },
  R6: {
    min_wallet_count:       3,
    min_wallet_percentile:  75,
    direction_threshold:    0.7,   // 70% of entrants must agree on direction
  },
  R9: {
    velocity_multiplier:    3.0,
    baseline_window_days:   7,
    min_notional:           5_000,
  },
};
```

Adding a Recipe

1. Add the function to `lib/signal-lab.ts` with signature `(pair: SnapshotPair) ⟹ SignalEvent[]`

2. Register it in the `recipes` map inside `runSignalLab()` with the next available ID (R14, R15, etc.)
3. Add display label and description to `lib/recipe-meta.ts` keyed by the recipe ID string
4. Add a config entry to `lib/recipe-config.ts`
5. Insert a row into `recipe_performance` so outcome tracking begins immediately

---

SECTION 05

Quality Assurance: Streams E-F

E Signal Validation

Stream E runs inside `/api/refresh-cohort` immediately after `runSignalLab()`. It discards signals that fail integrity checks before they enter the KV snapshot.

VALIDATION GATES

- **Wallet score gate:** wallet must have `score ≥ 50` at signal time
- **Active cohort gate:** `is_active = true` in the `wallets` table
- **Stream C gate:** `wash_score ≤ 0.3`, re-checked at signal time, not just at activation
- **Minimum notional:** signal position must be >= $500 notional to avoid dust position noise
- **Cooldown gate:** same recipe + wallet + coin combination cannot fire more than once per hour
- **EV floor:** signals with `ev_score < 0` are dropped

F Signal Learning

Stream F runs daily at 01:00 UTC via `scripts/signal-learning.ts`. It computes directional accuracy per recipe and updates `signal_outcomes` and `recipe_calibration`.

```
// scripts/signal-learning.ts (high-level flow)
1. Fetch signals from signals_history where ts > 24h ago
2. For each signal, fetch coin price at ts + 1h and ts + 4h
3. Classify outcome: WIN if price moved in signal direction
   by more than the threshold; LOSS otherwise
4. Upsert outcome row into signal_outcomes
5. Recompute win_prob per recipe over rolling 30-day window
6. Write updated win_prob to recipe_calibration
7. Upload learning-summary.json artifact (14-day retention)
```

Market Radar (E output)

The Market Radar view at `/signals/radar` is powered by `lib/radar-utils.ts`. It aggregates cohort position data per coin from the same KV snapshot as the main feed. No additional data fetching is required.

Key output fields per coin: `long_count`, `short_count`, `net_direction`, `total_notional_usd`, `top_wallets` (up to 5). The `cohort:underwater_counts` KV key provides the losing-position overlay.

Phase 2 Dependency

Auto-calibration in Phase 2 is gated on accumulating at least 100 entries in `signal_outcomes` and 30 days of data. Until that threshold is reached, `win_prob` values in `recipe_calibration` are seeded with conservative priors rather than empirical data.

SECTION 06

Infrastructure

GitHub Actions Workflows

DAILY-WALLET-SCAN.YML

A C D

- Schedule: `0 0 * * *` UTC; supports `workflow_dispatch`
- Timeout: 50 minutes
- Runs: `npx tsx scripts/daily-wallet-scan.ts`
- Required env: `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `HYPERLIQUID_API_URL`
- Artifact: `scan-summary.json`, 7-day retention

SIGNAL-LEARNING.YML

F

- Schedule: `0 1 * * *` UTC, after daily scan completes
- Timeout: 20 minutes
- Runs: `npx tsx scripts/signal-learning.ts`
- Artifact: `learning-summary.json`, 14-day retention

FRESHNESS-CHECK.YML

- Schedule: every 15 minutes (`*/15 * * * *`)
- Hits `/api/cohort-state` and fails if `updated_at` is older than 1200 seconds
- Emails repo admins on failure; catches silent cron outages before they surface to users

Vercel Configuration

Runs on the Vercel Hobby plan. The Vercel Cron pings `/api/refresh-cohort` daily at 00:00 UTC. The endpoint must complete in under 10 seconds; all heavy computation is offloaded to GitHub Actions. Upgrading to Pro removes this limit and would allow inlining the daily scan.

```
SUPABASE_URL=                    # Supabase project URL
SUPABASE_ANON_KEY=               # Browser-safe anon key (never used server-side)
SUPABASE_SERVICE_ROLE_KEY=       # Full access; never sent to browser
KV_URL=                          # Vercel KV connection URL
KV_REST_API_URL=                 # KV REST endpoint
```

```
KV_REST_API_TOKEN=               # KV read/write token
KV_REST_API_READ_ONLY_TOKEN=     # KV read-only token
HYPERLIQUID_API_URL=https://api.hyperliquid.xyz/info
NEXT_PUBLIC_POLL_INTERVAL_MS=60000
CRON_SECRET=                     # Optional; restricts cron to Vercel scheduler
```

SECTION 07

API Routes

ROUTE	METHOD	AUTH	DESCRIPTION
/api/refresh-cohort	GET/POST	CRON_SECRET (opt.)	Cron + manual trigger. Runs Stream B, Stream E, writes KV. Must finish in 10s on Hobby.
/api/cohort-state	GET	None	Primary client poll. Reads `cohort:active` KV. Fires background refresh via `after()` if stale > 5 min.
/api/contrarian	GET	None	Reads `contrarian:latest` KV. Fires background refresh if stale.
/api/market-ticker	GET	None	Live price and 24h change from `market-ticker:v4` KV.
/api/signals-feed	GET	None	Paginated signal event feed from `signals_history` Supabase table.
/api/wallet-profile	GET	None	Per-wallet deep dive: scores, positions, Stream D profile. Query: `?address=0x ...`
/api/recipe-performance	GET	None	Ranked recipe stats from `recipe_performance` table.
/api/market-radar	GET	None	Per-coin cohort positioning via `lib/radar-utils.ts`.
/api/scanner-stats	GET	None	Discovery stats: candidate count, gate pass rates, cohort size over time.
/api/top-markets	GET	None	Top coins by cohort notional and signal frequency.
/api/deep-dive	GET	None	Expanded wallet analysis including 30-day PnL chart data.
/api/measure-outcomes	POST	Internal	Called by signal-learning.ts to write outcome data to Supabase.

Stale-While-Revalidate Pattern

```
// /api/cohort-state (simplified)
const snapshot = await kv.get("cohort:active");

if (snapshot) {
  const ageSeconds = (Date.now() - snapshot.updated_at) / 1000;
  if (ageSeconds > 300) {
    after(async () => {
      await fetch("/api/refresh-cohort", { method: "POST" });
    });
```

```
  }
  return NextResponse.json(snapshot);
}

// Cache miss fallback chain
const fallback = await kv.get("cohort:active:fallback");
if (fallback) return NextResponse.json(fallback);
return NextResponse.json(await fetchFromSupabase());
```

Auth Model

Data routes have no OAuth. Server-side reads use `SUPABASE_SERVICE_ROLE_KEY`, which is never exposed to the browser. Client-side Supabase calls (if any) use the anon key. `CRON_SECRET` restricts the cron endpoint to Vercel's scheduler IP range when set.

SECTION 08

Known Limitations

VERCEL CRON BUDGET

The Hobby plan imposes a 10-second execution limit. `/api/refresh-cohort` cannot perform full cohort rescoring; it only runs signal detection and hygiene checks. Full scoring lives in the 50-minute GitHub Actions job. Upgrading to Pro would allow inlining the daily scan.

SIGNAL LEARNING DATA LAG

Phase 2 auto-calibration is blocked until 100+ signal outcomes accumulate in `signal_outcomes` and at least 30 days of data exist. Until that threshold is reached, `win_prob` values in `recipe_calibration` are conservative priors, not empirical data.

HYPERLIQUID API RATE LIMITS

The daily scan issues one `/info` POST per wallet for clearinghouse state and one per wallet for fill history. At 5,000 candidates that is 10,000 requests. A 50ms per-request delay keeps the scan within Hyperliquid's undocumented rate limit. If the limit tightens, the scan will fail with 429 errors and produce partial data in the artifact.

NO REAL-TIME POSITION UPDATES

Positions are polled, not streamed. The client poll interval is 60 seconds (`NEXT_PUBLIC_POLL_INTERVAL_MS`). Intraday entries and exits that resolve within a single 60-second window may be missed by recipe pairs that compare consecutive snapshots.

KV TTL RACE ON CRON MISS

If both `cohort:active` and `cohort:active:fallback` expire before a refresh cycle completes, the API falls back to a live Supabase query (200-500ms vs 5ms for KV). The freshness-check workflow emails on stale data older than 1200s to catch cron outages before this degrades the user experience.

SYBIL DETECTION COVERAGE

Stream C detects Sybil clusters via fill counterparty graph analysis. Wallets that use intermediate addresses to obscure coordination may not be flagged. Detection is probabilistic. Gate G8 uses a conservative cluster-size threshold to minimize false negatives at the cost of some false positives.

SECTION 09

Glossary

TERM	DEFINITION
`cohort`	The set of ~500 active wallets that have passed all Stream A gates and have not been deactivated by Stream B.
`SnapshotPair`	The prior and current cohort snapshots passed to each signal recipe for diff-based analysis.
`SignalEvent`	Typed event object emitted by a recipe; includes recipe_id, wallet, coin, direction, signal_type, ev_score, and timestamp.
`ev_score`	Expected value in USD of a signal, net of Hyperliquid fees and estimated slippage via walkBook.
`regime`	Market context derived from BTC 24h return: BULL (>+1%), BEAR (<-1%), or RANGING.
`Stream A`	Activation gate pipeline; 9-point quality check that admits wallets to the cohort.
`Stream B`	Hygiene layer; monitors active wallets and deactivates those that go quiet or blow up.
`Stream C`	Wash-trading and Sybil detection; writes wash_score and cluster_id to the wash_sybil table.
`Stream D`	Behavior profiling; classifies each wallet by trading style, conviction, and regime tendency.
`Stream E`	Signal validation; filters emitted signals before they enter the KV snapshot.
`Stream F`	Signal learning; computes outcome stats and updates recipe calibration daily.
`walkBook`	Slippage estimator in risk-engine.ts; simulates a market order fill against order book depth.
`recipe_calibration`	Supabase table storing per-recipe win_prob values updated by Stream F.
`wash_score`	0.0 to 1.0 score per wallet from Stream C; high value indicates self-trading patterns.
`regime_fit`	Scoring factor (weight 15%); wallet win rate in the current market regime.
`drawdown_score`	Scoring factor (weight 25%); 1 minus the ratio of equity drawdown from peak.
`pnl_consistency`	Scoring factor (weight 25%); winning-day ratio over a 30-day window.
`sharpe_proxy`	Scoring factor (weight 35%); mean daily PnL divided by standard deviation over 30 days.
`R1-R13`	Recipe IDs for the 13 signal recipes registered in lib/signal-lab.ts.
`after()`	Next.js API for fire-and-forget background execution after a response is sent; used for stale-while-revalidate refreshes.
`cohort:active`	Primary Vercel KV key holding the current scored cohort snapshot; TTL 120s.

TERM	DEFINITION
`cohort:active:fallback`	Backup KV key written alongside primary; TTL 24h; served when primary has expired.